Enhanced Optimal Multi-Row Detailed Placement for Neighbor Diffusion Effect Mitigation in Sub-10 nm VLSI

Changho Han, Andrew B. Kahng, *Fellow, IEEE*, Lutong Wang, *Student Member, IEEE*, and Bangqi Xu, *Student Member, IEEE*

***Abstract*—Layout-dependent effect causes variation in device performance as well as mismatch in model-hardware correlation in sub-10 nm nodes. In order to effectively explore the power-performance envelope for IC design, cell libraries must provide cells with different diffusion heights, leading to neighbor diffusion effect (NDE) due to inter-cell diffusion height change (diffusion *steps*). Special filler cells can protect against *steps* to functional cells, but with nontrivial area overhead. In this paper, we develop dynamic programming (DP)-based single-row and multi-row (MR) detailed placement optimizations that optimally[1] reduce inter-cell diffusion *steps* to mitigate the impacts of NDE. Compared to previous works, our algorithms are capable of exploring richer solution spaces as they support cell flipping, relocating, and reordering across cell rows; we also consider cell displacement, flipping, and wirelength costs. Notably, to our knowledge, our MR DP-based optimization algorithm is the first to optimally handle inter-row cell relocating and reordering. We also explore various metaheuristic configurations to further improve the solution quality. Last, we develop a timing-aware approach, which is capable of creating intentional *steps* that can potentially improve the drive strength of critical cells.**

***Index Terms*—Detailed placement, dynamic programming (DP), multi-row (MR) detailed placement, neighbor diffusion effect (NDE), timing-aware.**

I. Introduction

IN ADVANCED technology nodes, device behavior no longer depends on independent geometrical parameters [6]. Due to aggressive device scaling, lithography limitations and process complexity, layout-dependent effect (LDE) arises from the proximity of devices, and significantly affects device performance. An important type of LDE is neighbor diffusion effect (NDE) [1], where the horizontal spacing between diffusion regions changes the performance of transistors. Fig. 1(a) illustrates different diffusion spacing caused by diffusion height changes between four transistors. If the heights of neighboring diffusion regions are different, there is a diffusion *step*, e.g., transistor *T2* has a diffusion *step* to each of *T1* and *T3*.

More specifically, the drive strength (i.e., I_{on}) and the leakage power (i.e., I_{off}) of a transistor fin is a function of the horizontal spacing to the adjacent diffusion regions of the transistor fin. Since NDE changes the electrical characteristics of transistors, it affects the power, performance, and area of designs [1]. For example, Fig. 1(a) shows the transistor fins A and B with the spacings to their neighboring diffusion area, i.e., d_A and d_B, respectively. As d_A and d_B are different, I_{on} and I_{off} of the two transistor fins are different, e.g., $I_{\text{off}}(A) = f(d_A) \neq I_{\text{off}}(B) = f(d_B)$, due to the change in V_t [1]. For example, given a single inverter with a diffusion *step* next to the PFET and a diffusion *step* next to the NFET, the impacts to the two devices in combination result in higher leakage.

In this paper, we use a bimodal assumption to simplify the NDE problem: for a given transistor, either of two leakage values holds, depending on whether the diffusion region on the nearest neighboring site of the transistor has *full height* (that is, same or larger height), or *less height*, compared to

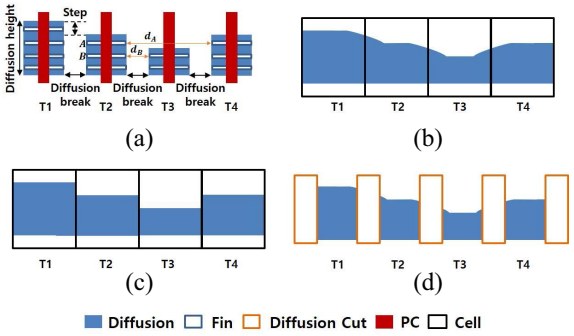


Fig. 1. (a) Diffusion *step* and fin spacing, (b) desired pattern, (c) actual diffusion region showing corner rounding, and (d) diffusion breaks (after diffusion cuts applied).

Manuscript received December 15, 2017; revised March 26, 2018 and June 6, 2018; accepted July 10, 2018. Date of publication July 24, 2018; date of current version August 20, 2019. This paper was recommended by Associate Editor I. H.-R. Jiang. *(Corresponding author: Lutong Wang.)*

C. Han is with the Design Technology Team (Foundry Business), Samsung Electronics Company, Ltd., Hwaseong 18448, South Korea (e-mail: changho1.han@samsung.com).

A. B. Kahng is with the Department of Computer Science and Engineering, University of California at San Diego, La Jolla, CA 92093 USA, and also with the Department of Electrical and Computer Engineering, University of California at San Diego, La Jolla, CA 92093 USA (e-mail: abk@ucsd.edu).

L. Wang and B. Xu are with the Department of Electrical and Computer Engineering, University of California at San Diego, La Jolla, CA 92093 USA (e-mail: luw002@ucsd.edu; bangqixu@ucsd.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCAD.2018.2859266

[1]The optimality is in terms of maximum diffusion *step* reduction, for given displacement range, reordering range and cell variants. Additionally, the above range definitions, including the definition of the ordering of cells, depend on assumptions described in Sections IV and V.

0278-0070 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

the transistor's diffusion height. The leakage difference for the above two cases is linear with #*steps*, e.g., a diffusion height difference of two steps results in 2× leakage difference compared to that of one step. In a conventional place-and-route flow, intra-cell NDE (i.e., NDE effect within a standard cell) is captured by library characterization since the diffusion shapes within a cell are predetermined. However, it is difficult to capture inter-cell NDE since neighboring diffusion shapes are determined by detailed placement. Thus, in general, library characterization always assumes existence of a full-height neighboring diffusion region on standard cell boundaries, which causes miscorrelation between the model (i.e., library) and the hardware (i.e., actual diffusion shapes at standard cell boundaries and their device leakage impacts) in a design. Minimizing diffusion *steps* in detailed placement is a key idea toward reduction of model-hardware miscorrelation.

With aggressive device scaling, the diffusion *step* not only causes NDE, but also induces an increase in the process complexity due to the limited resolution of conventional 193i lithography. In advanced nodes, the diffusion shapes of transistors are merged and patterned as a single polygon; the transistors are then separated by using diffusion breaks (which are achieved by applying diffusion cuts) [24], as shown in Fig. 1(a). Fig. 1(b) illustrates the desired pattern of a single polygon to generate the diffusion regions of four transistors. The actual pattern of the polygon (showing corner rounding in lithography) is shown in Fig. 1(c). Fig. 1(d) illustrates the final printed diffusion layout with diffusion cuts. At the boundaries of diffusion where diffusion *steps* exist, fin shapes and diffusion shapes are distorted due to the corner rounding phenomena. A distorted and/or sharp-angled end of a fin may cause an increase in electrical field, resulting in gate oxide breakdown [20]. Further, such distorted diffusion shapes change the diffusion height and fin length, which can cause dramatic shifts in threshold voltage (Vt), or even device failure in sub-10 nm nodes.[2] This Vt shift has negative impact on design performance and quality. For example, Vt variation can cause setup time and/or hold time violations in a design. As a result, the maximum frequency that the design can achieve is reduced, or the design can even fail with hold time violations due to ultralow Vt which cannot be recovered.

For a motivating study, we define a (inter-cell NDE-induced) *cell failure* to occur if the boundary transistor has a >100mV Vt shift compared to the average Vt for all transistors. According to [28], the failure rate of a transistor with a diffusion *step* is twice as high as a transistor without a diffusion *step* (base failure rate). The solid lines in Fig. 2 show the yield versus (initial) number of diffusion *steps* (∼ #cells) with different base failure rates. We assume #*steps* is approximately proportional to #cells, which holds for testcases in Section VI. The dashed lines in Fig. 2 show the projected yield for the same chip if we can reduce 90% of diffusion *steps*. In our preliminary study, more than 60% of standard cells (cell-boundary transistors) have inter-cell diffusion *steps*. For a relatively small design block VGA (85% utilization in a 7 nm design enablement, 69K cells and 50K diffusion *steps* initially), we assume a base failure rate of 1 ppm and can achieve 3.6% yield improvement by removing 90% of diffusion *steps*. For a commercial design with multiple hundreds of millions of cells and diffusion *steps*, if we assume a more realistic 1 ppb base failure rate, then we can achieve ∼3% yield improvement by removing 90% of diffusion *steps*.[3] In light of this, minimizing diffusion *steps* helps to recover the yield of designs by reducing Vt (and thus speed) variation of transistors.

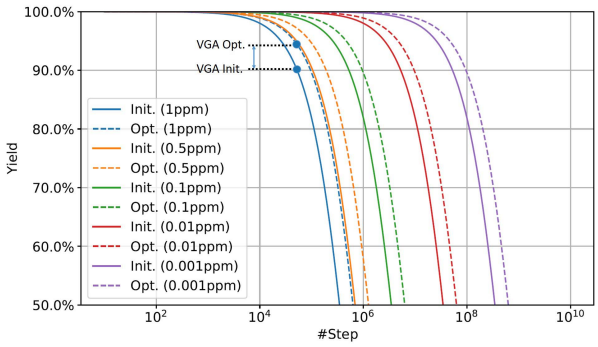


Fig. 2. Initial (Init.) and projected (Opt.) yield assuming 90% inter-cell *step* reduction for various base failure rate.

A. Current Limitations and Our Approach

In order to reduce diffusion *steps*, special nonfunctional *filler cells* are instantiated between functional cells [17] as we elaborate in Section III-A below. However, opportunities for *step*-reducing filler cell insertion are limited given a fixed layout, and this approach (effectively similar to cell padding) is expensive in terms of area. Other works [5], [16], [21], [26] propose graph-algorithmic or dynamic programming (DP) methods to resolve complex design rules in advanced nodes. However, the solution spaces considered are typically limited due to the assumption of (ordered)-single-row (SR) placement.[4] Recent works [15], [23] on multi-row (MR) detailed placement involve heuristic approaches, and no advanced-node rules are considered. Han *et al.* [7] proposed an optimal SR and double-row (DR) DP for detailed placement optimization, allowing cell reordering with support of double-height cells.

In this paper, we extend our previous SR and DR detailed placement framework [7] with HPWL-awareness and with MR detailed placement optimization. Our main contributions are summarized as follows.

1) We extend the optimal SR DP-based approach [7] to an HPWL-aware version. The proposed approach minimizes and balances diffusion *steps* and HPWL cost. Our proposed algorithm is capable of all types of cell movements—i.e., cell variants, relocating, and reordering (i.e., *P-reordering* with $P > 2$).

[2]According to our collaborator [28], there can be >150 mV Vt shift in the 10LPE node.

[3]Based on guidance from our collaborator [28], after scaling to account for our small testcase sizes, we assume a base failure rate of 1ppm for each *step* in our experiments with small design blocks. See Table IV in Section VI.

[4]Lin *et al.* [16] proposed a *P-reordering* problem. However, only *2-reordering* (i.e., neighbor cell switching) is presented. We describe our methodology to handle the *P-reordering* problem in Section III.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

2) We propose a new MR DP, with support of movable, and fully reorderable, multi-height cells, including reordering between multi-height cells. Inter-row cell moving within each optimization window (in multiple of rows) is intrinsically supported that further improves solution quality.
3) We propose metaheuristics to use both SR HPWL-aware optimization and MR optimization to achieve better solution quality.
4) We extend our formulation to a potential timing-aware optimization that leads to 6× increase in *intentional steps* around timing-critical cells to improve the timing performance.
5) We improve the solution quality over [7] by achieving up to 98% inter-cell diffusion *step* reduction compared to 90% achieved in [7], while consuming similar runtime.

The remainder of this paper is organized as follows. Section II reviews related works. Section III describes the problem formulation and DP-based SR detailed placement methodology. Section IV describes the DR detailed placement flow. Section V describes the MR detailed placement flow. In Section VI, we describe our experimental setup and results. Section VII gives conclusions and directions for ongoing work.

II. Previous Work

We classify relevant previous works on detailed placement into three categories: 1) detailed placement for advanced nodes; 2) mixed cell-height placement; and 3) NDE-aware detailed placement.

A. Detailed Placement for Advanced Nodes

To support complex design rules introduced in advanced nodes, the objectives of detailed placement have changed from classical objectives (e.g., wirelength reduction [9]–[11], [13], [14], [19]) in recent years. The works of [16], [21], and [26] resolve triple-patterning issues. Yu *et al.* [26] proposed shortest path and DP algorithms to solve the ordered-single-row (OSR) placement. Tian *et al.* [21] developed a weighted partial MAX SAT approach to solve the OSR problem. Lin *et al.* [16] proposed a local reordered single row refinement and implement a 2-reordering (i.e., neighboring cell switching) approach using a unified graph model. Du and Wong [5] applied a shortest-path algorithm supporting flipping and 2-reordering to address the drain-drain abutment problem in FinFET-based cell placement. The works of [3] and [8] propose mixed integer linear programming-based methods to comply with drain–drain abutment, minimum implant area, and minimum oxide jog length rules, and to increase vertical M1 connections.

B. Mixed Cell-Height Placement

Wu and Chu [23] proposed a pairing technique to handle double-height cells for detailed placement. Their method simply groups or inflates cells so that all cells become double-height cells, after which a conventional detailed placer can be used. Recently, Lin *et al.* [15] have proposed a *chain move* scheme along with a nested DP-based approach to support multiple cell-height placement. They first perform chain moves to save wirelength cost. On top of this, DP is applied to solve the nested shortest path problem. Other techniques [4] are developed to support noninteger-ratio (e.g., mixture of 8T and 12T cells) mixed cell-height placement.

C. NDE-Aware Placement

Ou *et al.* [18] performed NDE-aware analog placement by modifying and integrating a compact model for NDE into an existing analog placement algorithm. Oh [17] developed special filler cells to mitigate NDE.

Han *et al.* [7] (which this paper builds on) proposed to resolve the NDE problem in detailed placement stage. Inter-cell diffusion *steps* are minimized by trying to match the diffusion heights of neighboring cells. If two neighboring cells have different diffusion heights, special filler cells can be inserted to reduce diffusion *steps*. Han *et al.* [7] proposed SR and DR DP optimizations that support cell relocating, reordering and flipping as well as double-height cells. They support reordering between single-height cells, and between a single-height cell and a double-height cell, but not between two double-height cells.

In summary, many works such as [5], [16], [21], and [26] propose graph or DP models to resolve complex design rules in advanced nodes. However, their solution spaces are limited by the assumption of (ordered)-SR placement. Two recent works [15], [23] on MR detailed placement give heuristic approaches, but no advanced node rules are considered. Our previous work [7] proposes DP-based methods to optimize SR and DR placements, systematically supporting cell reordering and double-height cells. However, the DP formulation cannot be extended to support more than two rows, and the formulation cannot support reordering between two double-height cells. Notably, this paper advances over [7], and is distinguished from previous approaches, in several ways.

1) We formulate an optimal (HPWL-aware) SR and MR DP-based approach to minimize a cost function that includes diffusion *steps*.
2) We support a richer set of cell movements than in previous works—i.e., flipping, relocating, and reordering—via a systematic methodology to handle *P-reordering* with $P > 2$. Specifically, our MR approach intrinsically supports *inter-row* cell relocation.
3) Our formulation supports multi-height cells with movable, and fully reorderable, multi-height cells.

III. Single-Row Optimization

In this section, we describe the problem statement and our DP formulation for *single-row detailed placement*.

Single-Row Optimization Problem: Given an initial legalized SR placement, perturb the placement to minimize inter-cell diffusion *steps*.

Inputs: Legalized SR placement, available cell variants, and cost function of a diffusion *step*.

Output: Optimized SR detailed placement with minimized overall cost (including inter-cell diffusion *steps*).

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COST FOR ONE DIFFUSION *Step*

Spacing (sites)	0	1	2	3	4+
Cost	1	$+\infty$	1	1	0

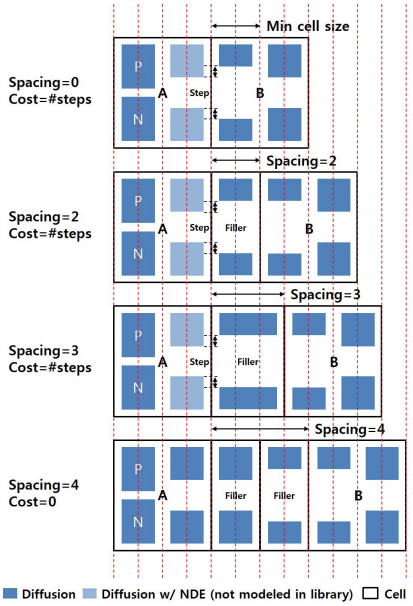


Fig. 3. Filler insertion between cell A and B, given different spacings.

Constraints: Maximum displacement range, maximum reordering range, and availability of cell flipping.

A. Filler Cell and Step Costs

Table I describes inter-cell diffusion *step* cost. For each pair of adjacent cells, if there are zero, two or three empty sites in between, the cost is equal to the number of inter-cell diffusion *steps*; if there are at least four empty sites in between, the cost is always zero. That is, with four or more empty sites we can always assume proper filler cell insertions resulting in no inter-cell diffusion *steps*. Fig. 3 shows an example of filler cell insertion between two functional cells that have different diffusion heights at edges that face each other. If the two functional cells have fewer than four empty sites in between, filler cells can only match one of the diffusion heights. As a result, there always exists at least one diffusion *step* that affects one of the two functional cells. However, with a spacing of four or more sites, a legal diffusion height transition can always be achieved by one or more contiguous filler cell(s). Thus, the filler cell(s) can match both the diffusion heights of the two functional cells. In a relevant advanced technology, the minimum filler cell width is two placement sites due to process limitations. Therefore, adjacent functional cells must abut, or have at least two empty sites between them, in order to insert a filler cell [28]. In our implementation, we avoid single-site spacings by assigning infinite cost to such scenarios, as indicated in Table I. Even though our optimization does not explicitly allocate white space, the DP (presented later in Sections III–V) itself can utilize/change the local white space distribution by cell relocating and cell reordering within specified ranges. Filler cell cost is explicitly included in our DP cost calculation, such that our optimization is aware of both whitespace and filler insertion as it trades off between: 1) abutting two cells without filler insertion at the cost of diffusion *steps* and 2) leaving at least four placement sites for a proper filler insertion in an effort to minimize the diffusion *steps* between neighboring cells.

TABLE II
NOTATIONS

Notation	Meaning
C	set of cells in a window of initial placement
c_k	k^{th} cell in the left-to-right ordered initial placement i.e., k is the cell index
v	a cell variant
$w_{k,v}$	width of c_k with a variant v
$[-x_\Delta, x_\Delta]$	horizontal displacement range
x_k	absolute x-coordinate of c_k in the initial placement, in units of placement sites
l	displacement from the initial placement, in units of placement sites
$[-r, r]$	reordering range
i	number of placed cells
j	position shift from the initial placement
s	placement status array
$d[i][j][v][l][s]$	minimum cost when i cells are placed with case (j,v,l,s)
The notations below apply only to multi-row optimization	
$[-y_\Delta, y_\Delta]$	vertical displacement range
y_k	absolute y-coordinate of c_k in the initial placement, in units of rows
m	number of rows in an optimization window
b	row index in an optimization window
d_b	for the bth row, d_b is the distance between the rightmost boundary of bth row, and the rightmost boundary of all rows in the optimization window
D	distance array of d_b in an optimization window (i.e. $[d_0 \ldots d_{m-1}]$)
t_b	for b^{th} row, type of the rightmost cell (e.g., 2-fin, 3-fin or 4-fin)
T	type array of t_b in an optimization window (i.e., $[t_0 \ldots t_{m-1}]$)
$\{D, T\}$	boundary condition
$[D][T]$	forming boundary condition $\{D, T\}$
$d[i][j][v][l][s]$ $[D][T]$	minimum cost when i cells are placed, forming boundary condition $\{D, T\}$

B. Notations

Table II shows notations used in our formulation. For each cell c_k, cell index k is its (left-to-right) sequentially ordered position in the initial placement. Given a set of cells (C) in a row of an initial placement, the leftmost cell is c_1, and the rightmost cell is $c_{|C|}$.

For each c_k, we define cell variants (v) which correspond to different cell orientations and cell layouts with the same functionality. To minimize #diffusion *steps*, we can use several variants of a cell with the same functionality, for which layouts have different diffusion heights. In our experiments below, $v = 0$ indicates the cell orientation in the initial placement, and $v = 1$ indicates the flipped (i.e., mirrored about the y-axis) cell orientation. $w_{k,v}$ is the width of cell c_k with variant v, in units of placement sites. Flipping a cell does not change the set of sites that the cell occupies.

We define the displacement range $[-x_\Delta, x_\Delta]$ as the constraint that a cell cannot move more than x_Δ sites from its initial placement. We use x_k to denote the initial right

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

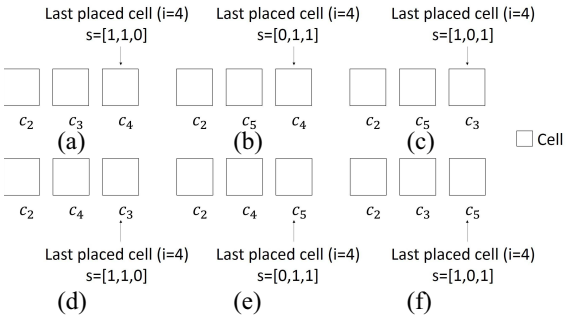


Fig. 4. Illustration of six placement solutions with three legal states given $i = 4$ and $r = 1$.

x-coordinate of c_k, in units of placement sites. Thus, c_k can be placed with its right x-coordinate in the interval $[x_k - x_\Delta, x_k + x_\Delta]$. We use l to denote the displacement (in sites) from the initial cell placement (i.e., $l \in [-x_\Delta, x_\Delta]$). For the cells on the boundary of the die, we make sure that the displacement range will not extend beyond the die boundary.

We support cell reordering with a reordering range $[-r, r]$, i.e., given r, in the placement solution c_k can have a new sequentially ordered position within the range $k - r, k - r + 1, \ldots, k + r$.

In our DP, we place one cell at a time from left to right, and the index i is used to indicate that i cells have been placed. Given a cell reordering range $[-r, r]$, cells c_k with $k < i - r$ are placed, $i - r \leq k \leq i + r$ may or may not be placed, and $k > i + r$ are not placed. For the $2r + 1$ cells such that $i - r \leq k \leq i + r$, we use a binary array s to denote the *placement status* of each cell. Here, s is a binary array of size $(2r + 1)$, i.e., $s \in \{0, 1\}^{2r+1}$. Each bit in the array indicates whether the corresponding cell is placed or not. For example, if we have six cells c_1 to c_6, $i = 4$ and $r = 1$, then s captures the placement status of the $(2 \times 1 + 1 = 3)$ cells c_3, c_4, and c_5. $s = [0, 1, 1]$ means that c_3 is not placed, while c_4 and c_5 are placed. Fig. 4 illustrates six placement solutions with three legal states when $i = 4$. In this example, c_1 and c_2 must be placed and c_6 must not be placed. We note that the indices of s correspond to k (position in the initial placement), but not the final position. For example, $s[0]$ always represents the status for c_3, and $s[2]$ always represents the status for c_5, regardless of the actual sequence of positions, as shown in Fig. 4(b). Also, when we have placed i cells, since cells with index $k < i - r$ must be placed, we must have placed $i - (i - r - 1) = r + 1$ cells that have cell index $i - r \leq k \leq i + r$. Thus, at all times, a legal status array s has exactly $r + 1$ elements equal to 1. In the above example, s always has $1 + 1 = 2$ elements equal to 1.

Given i, to identify the last placed cell c_k (that is, the ith cell to have been placed), we define the position shift as j, where $k = i + j$. For example, in Fig. 4(c), given $i = 4$, the position shift $j = -1$ tells that the last placed cell is c_3, since $3 = 4 + (-1)$.

At the heart of our DP recurrence, we use $d[i][j][v][l][D][T][s]$ to represent the minimum cost when i cells have been placed. Note that in SR case, the dimensions of D and T are both zero. Therefore, the DP array can be reduced to $d[i][j][v][l][s]$. From this array, we can obtain the last placed cell c_k, where $k = i+j$. We can also tell the variant v in use, the displacement l, and the status s for cell c_k. We define the above as *case* (j, v, l, s), with i implicitly given, for simplicity. Therefore, we complete the row placement once we reach $i = |C|$, and we obtain the optimal solution by finding the minimum cost among all cases of $i = |C|$. In our implementation, we store a pointer for each entry in the DP array so that the optimized placement can be traced back from $d[|C|][j][v][l][s]$ all the way to $d[0][j][v][l][s]$.

Algorithm 1 DP (SR)

```
1: Initialize for all legal cases (j, v, l, s)
2:    d[0][j][v][l][s] ← 0, d[i][j][v][l][s] ← +∞, (0 < i ≤ |C|)
3: for all i = 0 to |C| − 1 do
4:    for all d[i][j][v][l][s] ≠ +∞ do
5:       for all (j′, s′) ∈ getNext(s) do
6:          for all (v′, l′) do
7:             i′ = i + 1
8:             t ← d[i][j][v][l][s] + cost(i,j,v,l,i′,j′,v′,l′)
9:             d[i′][j′][v′][l′][s′] ← min (d[i′][j′][v′][l′][s′], t)
10:         end for
11:      end for
12:   end for
13: end for
14: finalCost ← ∞
15: for all (j, v, l, s), i = |C| do
16:    finalCost ← min (d[|C|][j][v][l][s], finalCost)
17: end for
18: Return finalCost
```

C. Dynamic Programming Formulation

Algorithm 1 describes our DP procedure for SR placement in detail. Line 2 initializes the DP solution array. Lines 3–13 describe the main algorithm. Starting with placing the first cell, the algorithm incrementally adds (places) cells next to the current partial placement solution. Procedure *getNext*() returns a list of legal *next* cells and the respective status of each of these cells. Along with legal (j', s') from line 5, line 6 checks all possible cases (v', l') considering placement legality and displacement constraints, as shown in (1). Lines 7–9 update the minimum cost for the case (j', v', l', s') when we place the $i' = (i+1)st$ cell. In lines 14–17, we obtain the minimum cost among all legal cases when $i = |C|$, and line 18 returns the minimum cost for the current row

$$x_{i+j} + l + w_{i+j,v} \leq x_{i'+j'} + l'. \tag{1}$$

The function $\text{cost}(^{i',j',v',l'}_{i,j,v,l})$ calculates the cost as a weighted sum of: 1) diffusion *step* cost; 2) displacement cost; and 3) cell variant cost, as shown in (2). The diffusion *step* cost is calculated as total #inter-cell diffusion *steps* between the ith and (i')th placed cells. The displacement cost is equal to the absolute value of l'. In this paper, we assume that the given initial placement solution has adequate quality in terms of various metrics, including but not limited to pin accessibility, global routability, etc. Thus, we simplify other optimization objectives as one "displacement minimization" objective. As noted above, in this paper we assume two cell variants: 1) original orientation and 2) flipped orientation. We set the variant cost to one if a cell is flipped ($v' = 1$), and zero otherwise. Two weighting factors α and β (β can be seen as supplementing α by capturing an equivalence between cell flipping

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Procedure *getNext* (SR)

```
1: Inputs: s
2: Initialize nextList ← ∅
3: s ← shiftLeft1Bit(s)
4: if s[−r] = 0 then
5:     s[−r] ← 1
6:     nextStatus ← s
7:     nextList ← nextList ∪ {(−r, nextStatus)}
8:     Return nextList
9: end if
10: for all m ∈ [−r, r] do
11:     if s[m] = 0 then
12:         nextStatus ← s
13:         nextStatus[m] ← 1
14:         nextList ← nextList ∪ {(m, nextStatus)}
15:     end if
16: end for
17: Return nextList
```

and displacement) are used to balance the three cost terms. We describe experiments regarding the impact of weighting factors in Section VI

$$\text{cost}\binom{i',j',v',l'}{i,j,v,l} = \text{cost}_{\text{step}} + \alpha \cdot \text{cost}_{\text{disp}} + \alpha \cdot \beta \cdot \text{cost}_{\text{var}}. \quad (2)$$

Algorithm 2 details our methodology to obtain *next* status. That is, given the binary status array for i, we construct the status array for $i' = i + 1$. Line 2 initializes the list of next available (*cellIndex*, *status*) combinations. In line 3, we first shift the status array for i one bit to the left to obtain the cell placement status for $i' = i+1$. Then, lines 4–9 check whether cell $c_{i'-r}$ must be placed as the (i')th cell. If we do not place $c_{i'-r}$ as the (i')th cell, then cell $c_{i'-r}$ will be placed out of its reordering range. Thus, we set $s[-r] = 1$ and return so that we make sure to choose $c_{i'-r}$ as the (i')th cell. Lines 10–16 check whether any binary indicator $s[m]$ is equal to zero. If so, $c_{i'+m}$ could be the next legally placed cell. In such a case, we add $(m, nextStatus)$ to the list.

D. HPWL-Aware Optimization

We mitigate the wirelength impact of SR *step* optimization by modifying the cost function. Specifically, we add a ΔHPWL cost component to the function $\text{cost}\binom{i',j',v',l'}{i,j,v,l}$ as

$$\text{cost}\binom{i',j',v',l'}{i,j,v,l} = \text{cost}_{\text{step}} + \alpha \cdot \text{cost}_{\text{disp}} + \alpha \cdot \beta \cdot \text{cost}_{\text{var}} + \gamma \cdot \text{cost}_{\Delta\text{HPWL}}. \quad (3)$$

We calculate the $\text{cost}_{\Delta\text{HPWL}}$ by summing up the ΔHPWL contribution of cell c_k over all nets incident to c_k, in the same way as in [13]. $\text{cost}_{\Delta\text{HPWL}}$ captures the impact of a cell's placement on bounding box sizes of incident nets. We use a new weighting factor γ to balance the four cost terms. We describe experiments regarding the impact of weighting factors in Section VI.

IV. Double-Row Optimization

In this section, we briefly revisit the *double-row detailed placement* optimization presented in [7]. We give the problem statement addressed in [7], and explain limitations in DR optimization compared to the MR optimization that we describe below in Section V.

Double-Row Optimization Problem: Given an initial legalized DR placement with double-height cells, perturb the placement within each row to minimize inter-cell diffusion *steps*.

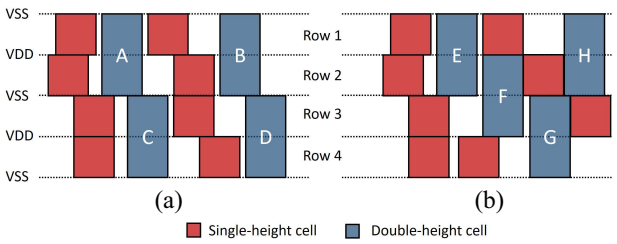


Fig. 5. Illustrations of double-height cells in placement rows. (a) Separable pairs of cell rows, reflecting power rail design of double-height cells in current N10 libraries. (b) Nonseparable pairs of cell rows.

Inputs: Legalized DR placement, available cell variants, and cost function of a diffusion *step*.

Output: Optimized DR detailed placement with minimized overall cost (including inter-cell diffusion *steps*).

Constraints: Maximum displacement range, maximum reordering range, availability of cell flipping.

We note the following two limitations with regard to this problem statement.

Limitation 1: Single-height cells cannot move across rows. In DR optimization, the double-height cell effectively breaks the two rows into separate optimization regions, wherein we invoke SR optimization separately for each of the two rows. Thus, in the DR optimization of [7], single-height cells cannot be relocated to the other row.

Limitation 2: The relative positions among double-height cells are fixed.

For two double-height cells A and B, if A is initially to the left of B ($x_A < x_B$), then we require that in our final placement, c_A remains to the left of c_B. We note that we still allow reordering between a single-height cell and a double-height cell (thus, the double-height cells are *partially reorderable*) so as to maximize the *steps* reduction.

In the next section, we describe a more generic MR optimization without the above limitations. Due to the above limitations compared to the MR optimization described in Section V, as well as the page limit, we omit further details. The complete DR formulation can be found in [7].

V. Multi-Row Optimization

In this section, we generalize from the SR DP, and describe our approach for *multi-row detailed placement*, with support of fully reorderable multi-height cells and inter-row cell relocating.

Multi-Row Optimization Problem: Given an initial legalized MR placement, perturb the placement across the multiple rows to minimize inter-cell diffusion *steps*.

Inputs: Legalized MR placement, available cell variants, and cost function of a diffusion *step*.

Output: Optimized MR detailed placement with minimized overall cost (including inter-cell diffusion *steps*).

Constraints: Maximum horizontal displacement range, maximum vertical displacement range, maximum reordering range, and availability of cell flipping.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

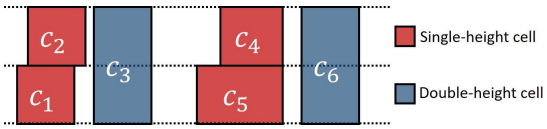


Fig. 6. Example of MR cell ordering. Cells are sequentially ordered (c_1 to c_6) according to the x-coordinate of their right boundary. Cells c_4 and c_5 have the same right boundary x-coordinate, and thus could be switched in the ordering.

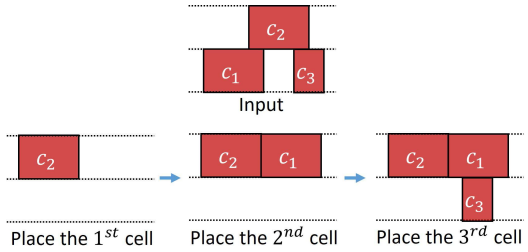


Fig. 7. Illustration of the Assumption.

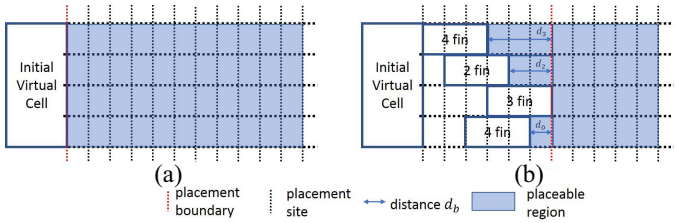


Fig. 8. Illustration of DP in MR placement with $m = 4$.

A. Preliminaries

Similar to DR optimization, we optimize m consecutive rows together (as a single optimization *window*) in MR optimization. In an optimization window, we move the cells according to our algorithm assuming that cells outside the window are fixed. Different windows are optimized separately. However, compared to the DR optimization in Section IV, we do not require the relative positions among double-height cells to be fixed. Instead, a double-height cell can be reordered with another double-height cell as long as they are within the reordering range. Moreover, in contrast to Section IV's DR optimization, where a cell cannot move outside its original cell row, here we allow a cell to move freely within a given vertical displacement range (in units of placement rows), enabling larger solution space to minimize diffusion *steps*.

In SR and DR optimization, where only intra-row relocating and reordering are allowed, the initial cell ordering (c_k in Table II) is defined within each row from the initial (input) placement. To enable a unified MR reordering range, with support of inter-row relocating and reordering, we redefine the original cell ordering as follows:

Definition: Given an m-row initial (input) placement, cells in all m rows are left-to-right ordered according to their rightmost boundary, in a unified 1-D array, e.g., $c_1, c_2, \ldots, c_k$. If cells in the initial placement have the same x-coordinate for their right boundary, we break ties using y-coordinate of their lower boundary.

Fig. 6 shows an example of sequential cell ordering for a two-row initial placement. We note that cells c_4 and c_5 could have their positions exchanged in the ordering, regardless of their left boundary. However, as mentioned, in our implementation tie-breaking is by descending order of y-coordinate.

With the above redefined cell ordering, reordering range works the same way as in Section III. The new sequentially ordered position is determined by the new x-coordinate (in the final solution) of the right boundary of each cell. The difference between the original and the new sequentially ordered position should be always within the reordering range. In the MR optimization, given the above redefined cell ordering, our DP still seeks to place one cell at a time, from left to right. The left-to-right placement procedure then induces the following assumption.

Assumption: The x-coordinate of the right boundary of the $(i+1)$st cell must be greater than or equal to the right boundary of the partial placement consisting of i cells (i.e., placement boundary).

Given the definition, the assumption does not reduce the solution space. For example, in Fig. 7, assuming a partial placement of c_2 and c_1, if the third cell to be placed is c_3, and we would like its right boundary to be to the left of the placement boundary, then we can always get to such a partial placement solution from a partial placement of c_2 and c_3, followed by placement of c_1.

B. Formulation

Given the above assumption, our approach will find an optimal placement solution for a given optimization window of m rows containing multi-height cells. We illustrate the MR DP-based detailed placement in Fig. 8(a). We use type array $T = \{t_0, \ldots, t_{m-1}\}$ to describe the type, i.e., 2-fin, 3-fin or 4-fin configuration, of the rightmost cell in each row. Initially, each entry of T is an initial virtual cell, indicating that the placement boundary for all rows is the left boundary of the die, and that there will be no diffusion *step* penalty applied to any type of cell immediately to the right of this boundary. We also use distance array $D = \{d_0, \ldots, d_{m-1}\}$ to describe the shape of the placeable region as shown in Fig. 8(b). The subproblems solved in the DP are of form: place $|C| - i$ cells in the placeable region defined by a partial placement with i cells.

We give a precise description of our MR DP in Algorithm 3. Note that the numbers of entries of distance array D and cell type array T are both $m-1$ because the distance from the last placed cell to the placement boundary is always zero, and the cell type of the last placed cell can be retrieved by cell variant v. Lines 1–3 initialize the DP solution array. Lines 4–15 describe the main algorithm. Compared to SR DP, we have one more iteration over all placement rows in an optimization window, subject to the maximum vertical displacement range constraint. Effectively, the MR DP array is different from SR DP array in that it is capable of storing multiple intermediate placement solutions given the same cell ordering and horizontal displacement, as long as these solutions have different type (T) or distance (D) arrays. Also, line 11 updates distance

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 DP (MR)

1: **Initialize costs for all legal cases** (j, v, l, b, D, T, s)
2: $d[0][j][v][l][b][D][T][s] \leftarrow 0$,
3: $d[i][j][v][l][b][D][T][s] \leftarrow +\infty, (0 < i \leq |C|)$
4: **for all** $i = 0$ to $|C| - 1$ **do**
5: **for all** $d[i][j][v][l][b][D][T][s] \neq \infty$ **do**
6: **for all** $(j', s') \in$ getNext(s) **do**
7: **for all** (v', l', b') **do**
8: $i' = i + 1$
9: $t \leftarrow d[i][j][v][l][b][D][T][s] + cost^{j',v',l',b'}_{i,j,v,l,b,D,T}$
10: $d[i'][j'][v'][l'][b'][D'][T'][s'] \leftarrow$ $\min(d[i'][j'][v'][l'][b'][D'][T'][s'], t)$
11: *Update*(D, T)
12: **end for**
13: **end for**
14: **end for**
15: **end for**
16: *finalCost* $\leftarrow \infty$
17: **for all** (j, v, l, b, D, T, s) **when** $i = |C|$ **do**
18: *finalCost* $\leftarrow \min(d[|C|][j][v][l][b][D][T][s], finalCost)$
19: **end for**
20: **Return** *finalCost*

array D and cell type array T according to the choice of placement row b'. Lines 16–19 obtain the optimal solution among all legal cases when $i = |C|$, and line 20 returns the optimal solution for the current optimization window.

MR optimization is not capable of being aware of HPWL change in y direction and across different optimization windows. Therefore, to prevent HPWL degradation, we add additional displacement costs if a cell is moved out of the original HPWL bounding box, with penalty coefficient γ_{penalty}, as shown in (4).[5] The term $\text{cost}_{\text{hpwl}}$ is calculated as the distance between the current cell and the original HPWL bounding box, in units of placement sites

$$\text{cost} = \text{cost}_{\text{step}} + \alpha \cdot \text{cost}_{\text{disp}} + \gamma_{\text{penalty}} \cdot \text{cost}_{\text{hpwl}} + \alpha \cdot \beta \cdot \text{cost}_{\text{var}}. \quad (4)$$

VI. Experiments

We implement our DP in C++ with OpenAccess 2.2.43 [31] to support LEF/DEF [30], and with OpenMP [33] to enable thread-level parallelism. We perform experiments in an N7 FinFET technology with multi-height triple-Vt libraries from a leading technology consortium. The fin height information is not disclosed in our enablement. Therefore, following guidance from [28], we randomly assign fin heights (2, 3, or 4 fins) to each cell with 1:3:6 ratio for 2, 3, and 4 fins, respectively, as our default fin height assignment methodology to match industrial designs at advanced nodes. For example, a double-height cell will have four random fin heights, i.e., for its left and right boundaries on the first row, and its left and right boundaries on the second row. Section VI-C further discusses the impact of alternative fin height assignment methods.

We generate the bimodal leakage values from the NDE-oblivious standard-cell Liberty file as follows [28]. Since NDE only affects the boundary transistors for each cell, given a leakage value of each standard cell from the Liberty file, we first approximate the boundary transistor leakage value by dividing the state-independent cell leakage by the cell width (in units of contacted-poly pitch), e.g., if a cell (width = 3) has a leakage value of three, then the boundary transistors have a leakage value of one. Then, for each diffusion *step*, 52% of boundary transistor leakage value is added to the cell leakage. In the above example, the cell has a new leakage value of 3.52 (resp. 4.04) when there exists one *step* (resp. two *steps*).

[5]We precalculate all net bounding boxes (one-time effort) and only apply the HPWL penalty if a cell is placed outside of its nets' bounding boxes.

TABLE III
Design Information

design	#inst	clkp
AES	∼12K	500ps
M0	∼10K	500ps
JPEG	∼54K	500ps
VGA	∼69K	500ps
MPEG	∼14K	500ps

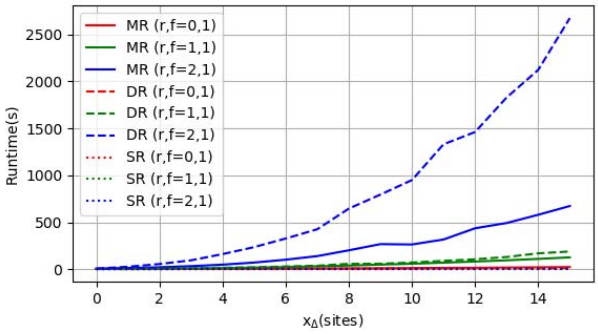


Fig. 9. Sensitivity of runtime to (x_Δ, r, f) parameters.

We apply our detailed placement optimization to Arm Cortex-M0 and four design blocks (AES, JPEG, VGA, and MPEG) from OpenCores [32]. Design information is summarized in Table III. We synthesize designs using *Synopsys Design Compiler L-2016.03-SP4* [34], and perform place-and-route using *Cadence Innovus Implementation System v15.2* [29]. We also apply our detailed placement optimization to winning solutions from the ICCAD-2017 multideck standard cell legalization contest [2]. All experiments are performed with eight threads on a 2.6 *GHz* Intel Xeon server.

In the following, we show: 1) the scalability and sensitivity, i.e., impact of cell displacement range x_Δ, reordering range r, enabling of cell flipping f, and #rows per window m for the MR implementation on runtime and quality of results (QoR in terms of *step* reduction); 2) impact of the weighting factors, i.e., weighting factor α for cell displacement, weighting factor β for cell flipping, and weighting factor γ for HPWL on QoR; 3) metaheuristics by combining SR HPWL-aware and MR optimization; 4) our main results with SR, DR, and MR optimization for five design blocks and three fin height assignment methodologies; 5) performance improvement using *intentional steps*; and 6) our results with MR optimization for ICCAD-2017 benchmark [2].

A. Scalability/Sensitivity Study

In this section, we compare the impact of reordering range and displacement range on the SR, DR and MR optimization. By default, we use $m = 2$ in MR optimization (see Fig. 10 and discussion below). Following the results of [7], cell flipping is enabled by default for maximum *step* reduction.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

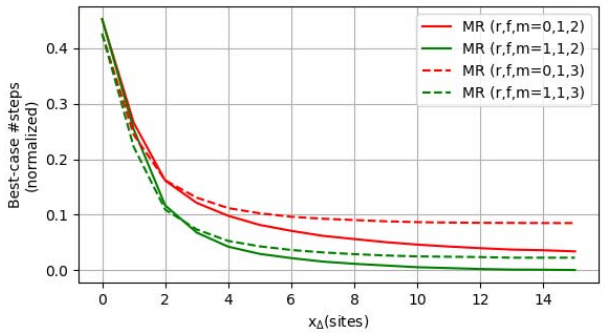


Fig. 10. Sensitivity of #*steps* to m in MR optimization.

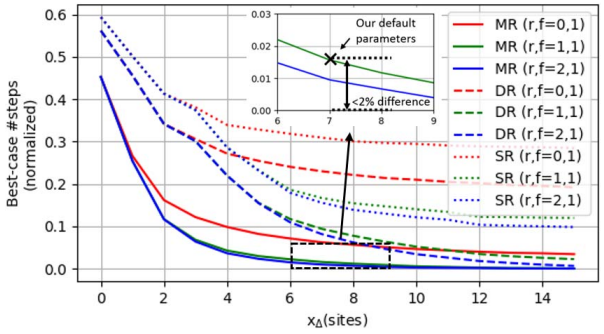


Fig. 11. Sensitivity of #*steps* to (x_Δ, r, f) parameters.

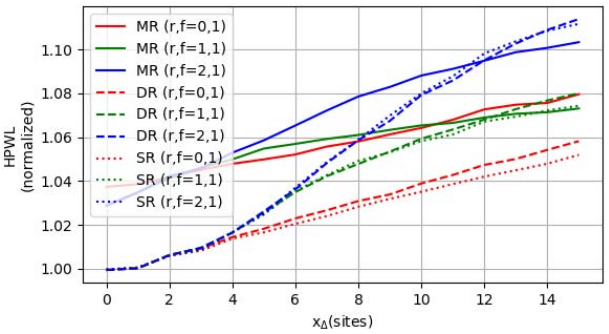


Fig. 12. Sensitivity of HPWL to (x_Δ, r, f) parameters.

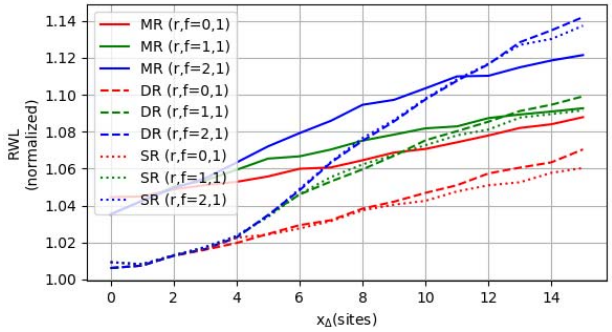


Fig. 13. Sensitivity of RWL to (x_Δ, r, f) parameters.

To assess the scalability of our approach, we sweep (x_Δ, r), i.e., maximum allowed cell displacement x_Δ (in placement sites) and maximum allowed one-sided reordering r, and study the impact on runtime. In this experiment, we sweep x_Δ from 0 to 15, and r from 0 to 2. A cell can freely move across 31 placement sites, and can have up to five different positions in a placement window, if we set $x_\Delta = 15$ and $r = 2$. We set $(\alpha, \beta) = (0, 0)$ as these parameters do not have any impact on the complexity of our formulation. We use design block AES for this study.[6]

The results are shown in Fig. 9. We find that the runtime generally grows quadratically with the number of available placement sites per each cell. However, for cell reordering, there is a dramatic increase in runtime as r goes up, e.g., we observe 12× runtime increase going from $r = 1$ to $r = 2$.

Also, compared to DR [7], our new MR implementation with $m = 2$ rows per window is much more efficient in terms of runtime. To investigate the impact of m (#rows in a window) in MR, we compare the sensitivity of #*steps* in Fig. 10 for $m = 2$ and $m = 3$. Runs with $m = 4$ are not feasible due to much larger memory consumption. We find that $m = 2$ actually gives better #*steps* than $m = 3$ using our N7 library, because all multi-height cells have VSS power rails for their cell boundaries, such that all multi-height cells are aligned per two cell rows. Given the above observation, we use $m = 2$ for MR in all of the following experiments.

To assess the sensitivity to (x_Δ, r), Figs. 11–13 show #diffusion *steps*, HPWL, and RWL, respectively, as we sweep (x_Δ, r). Since our algorithm only optimizes #diffusion *steps* when $(\alpha, \beta) = (0, 0)$, here we see HPWL and RWL that correspond to a best-case (minimized) #*steps* normalized to initial design.

We see from Fig. 11 that SR can only reduce #*steps* by up to 80%, while DR and MR are able to reduce #*steps* by up to 99% given larger displacement range. Also, MR is consistently better than DR, especially given a smaller displacement range. Along with the runtime benefit of MR, we believe that the new MR implementation surpasses both the solution quality and the runtime efficiency of DR [7].

[6]To investigate the stability of our sensitivity studies and observations, we also use 1) an alternative AES design implementation with slightly different layout and 2) design block CORTEXM0DS. Results for 1) and 2) are consistent with the results that we report here.

Moreover, for $f = 1$, there is only ∼0.6% benefit of using $r = 2$ over $r = 1$, at the cost of 12× the runtime; this suggests that $r \geq 2$ may not offer significant benefit in reducing #*steps*. In Figs. 12 and 13, HPWL and RWL increase linearly as x_Δ goes up. Based on these studies, to balance solution quality and runtime we apply $(x_\Delta, r) = (7, 1)$ in all of the following experiments.

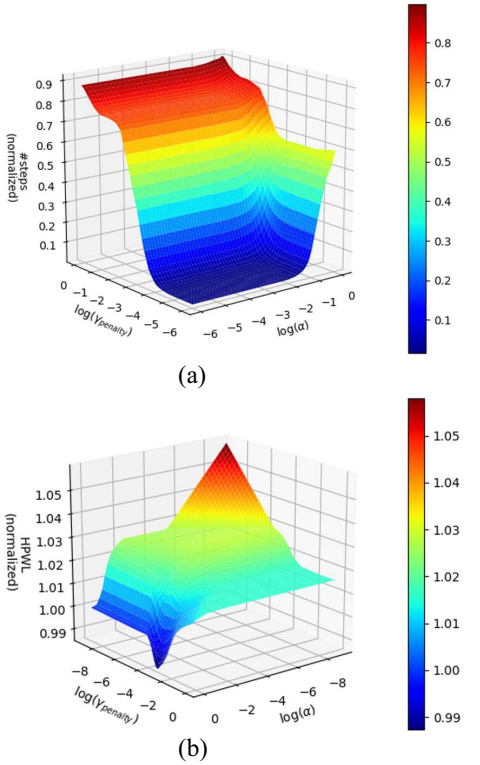
B. Study of Weighting Factors

In the following section, our default flow is MR optimization, with two rows per window. We investigate impacts of the weighting factors $(\alpha, \gamma_{\text{penalty}})$ for cell displacement and HPWL penalty on HPWL and #*steps*. We sweep α and γ_{penalty} from 0 to 1. We perform this experiment using design block AES. The results are shown in Fig. 14. We can see that a nonzero displacement weight (α) and a nonzero HPWL penalty $(\gamma_{\text{penalty}})$ save HPWL while preserving most of the

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EXPERIMENTAL RESULTS FOR ALL DESIGN BLOCKS USING MR OPTIMIZATION

Design	Type	Fin Height Distribution			#*steps*		RWL (μm)		WNS (ns)		Leakage (mW)		Runtime	Est. Yield
		2 fin%	3 fin%	4 fin%	Init	Final (Δ%)	Init	Final (Δ%)	Init	Final	Init	Final (Δ%)	(sec)	Impr. %
AES	rand	10.0	30.4	59.6	7973	152 (-98.1%)	31873	32995 (+3.5%)	-0.013	-0.021	16.1	15.8 (-2.1%)	162.1	+0.71
	Vt	48.3	47.8	3.9	6816	143 (-97.9%)	31874	32944 (+3.4%)	-0.013	-0.020	16.6	15.8 (-4.9%)	81.5	+0.66
	drive	47.5	46.6	5.9	7215	236 (-96.7%)	31874	32888 (+3.2%)	-0.013	-0.018	16.1	15.8 (-2.0%)	109.9	+0.69
M0	rand	10.1	30.4	59.4	6588	243 (-96.3%)	27670	28728 (+3.8%)	-0.043	-0.070	18.9	18.6 (-1.9%)	174.4	+0.22
	Vt	49.3	48.6	2.1	5379	152 (-97.2%)	27674	28588 (+3.3%)	-0.043	-0.111	19.5	18.6 (-4.5%)	74.1	+0.52
	drive	46.3	45.6	8.0	6211	398 (-93.6%)	27669	28718 (+3.8%)	-0.043	-0.051	19.1	18.6 (-2.6%)	64.5	+0.58
JPEG	rand	10.0	30.0	60.0	34760	656 (-98.1%)	101000	107699 (+6.6%)	-0.319	-0.278	96.3	94.3 (-2.1%)	776.5	+3.50
	Vt	48.2	48.6	3.3	29452	387 (-98.7%)	100997	106972 (+5.9%)	-0.319	-0.274	98.8	94.4 (-4.4%)	403.2	+2.78
	drive	44.0	44.5	11.5	36173	1291 (-96.4%)	101003	108103 (+7.0%)	-0.323	-0.290	97.2	94.4 (-2.9%)	398.2	+3.30
VGA	rand	10.0	30.1	60.0	50766	6179 (-87.8%)	208155	217492 (+4.5%)	-0.137	-0.080	208.3	205.1 (-1.5%)	713.3	+4.56
	Vt	48.8	49.6	1.6	40743	3685 (-91.0%)	208155	216603 (+4.1%)	-0.137	-0.069	213.4	205.5 (-3.7%)	536.8	+3.48
	drive	42.1	42.8	15.1	57273	10871 (-81.0%)	208155	217664 (+4.6%)	-0.137	-0.129	208.2	205.1 (-1.5%)	491.1	+4.24
MPEG	rand	9.9	30.5	59.6	9994	1367 (-86.3%)	38896	40594 (+4.4%)	-0.005	-0.018	33.2	33.1 (-0.2%)	137.3	+0.87
	Vt	49.6	49.4	1.0	7824	753 (-90.4%)	38882	40383 (+3.9%)	-0.011	-0.026	33.2	33.1 (-0.3%)	68.6	+0.70
	drive	43.1	43.1	13.8	10931	2145 (-80.4%)	38901	40649 (+4.5%)	-0.005	-0.030	33.2	33.1 (-0.3%)	99.5	+0.86



(a)

(b)

Fig. 14. Impacts of weighting factors (α, γ_{penalty}) on the tradeoff between HPWL and #*steps*.

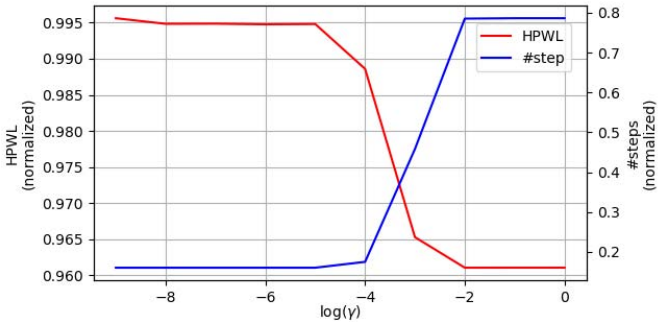


Fig. 15. Impact of weighting factor γ on the tradeoff between HPWL and #*steps*.

step reduction benefits. Therefore, we apply $\alpha = 0.01$ and $\gamma_{\text{penalty}} = 0.00001$ in all following experiments.

For the SR optimization, we also study the impact of the HPWL weighting factor γ on HPWL and #*steps*. We sweep γ from 0.00001 to 1 with a step size of 10×. We perform this experiment using design block AES, with results shown in Fig. 15. The tradeoff between HPWL and #*step* is clear when γ is in the range of $[0.00001, 0.01]$. We use $\gamma = 0.0001$ for the HPWL-aware SR optimization.

C. Main Results

We apply our MR DP-based optimization to all our design blocks using the aforementioned parameter settings, i.e., $(x_\Delta, r, f) = (7, 1, 1)$ and $(\alpha, \beta) = (0.01, 1)$. Table IV shows the *step* reduction, runtime, and estimated yield improvement for all five design blocks using MR optimization. We also report the impact on other metrics, i.e., routed wirelength (RWL), worst negative slack (WNS), and leakage power as reported by the place-and-route tool [29].

We also investigate the impact of fin height assignment methodologies. We apply three methodologies.

1) *rand* randomly assigns fin heights according to probability ratio 1:3:6 for 2, 3, and 4 fins, respectively, (see Section VI above).
2) *Vt* assigns fin heights according to their Vt property, with HVT (resp. NVT and LVT) cells having probability ratio 1:1:0 (resp. 1:1:1 and 0:1:1) for 2, 3, and 4 fins.
3) *drive* assigns fin height according to their drive strength, with X0 (resp. X1 and others) cells having probability ratio 1:1:0 (resp. 1:1:1 and 0:1:1) for 2, 3, and 4 fins.

The three methodologies generate different fin height distributions, and thus help confirm the robustness of our optimization in broader scenarios. The results are shown in Table IV. For all designs with the default (*rand*) distribution, we achieve up to 98.1% reduction in #*steps* at the cost of around 3.5% RWL increase. The results also show that our optimization has negligible impact on WNS and that we can slightly improve the leakage. In addition, we perform a preliminary yield estimation assuming 2 ppm failure rate for each *step*, and 1 ppm failure rate after we remove the *step* (recall Footnote 3). Based on this assumption, we can see a yield improvement of up to 4.56% for a design block of 69K instances. We note that the yield improvement is expected to grow markedly with the die

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF DIFFUSION *Steps* WITH SR (TO MATCH [5], [16]), ODR (TO MATCH [15]) DR, MR, AND METAHEURISTICS (META). DH%:= % OF DOUBLE-HEIGHT CELLS

Design	DH%	Init	SR (to match [5][16])	ODR (to match [15])	DR	MR	Meta
AES	4.3%	7973	1395 (-82.5%)	1869 (-76.6%)	750 (-90.6%)	152 (-98.1%)	131 (-98.4%)
M0	8.4%	6588	1672 (-74.6%)	1742 (-73.6%)	842 (-87.2%)	243 (-96.3%)	179 (-97.3%)
JPEG	8.3%	34760	9731 (-72.0%)	8341 (-76.0%)	4555 (-86.9%)	656 (-98.1%)	473 (-98.6%)
VGA	24.8%	50766	27170 (-46.5%)	16405 (-67.7%)	11816 (-76.7%)	6179 (-87.8%)	5652 (-88.9%)
MPEG	23.0%	9994	5101 (-49.0%)	3444 (-65.5%)	2402 (-76.0%)	1367 (-86.3%)	1215 (-87.8%)
Avg.	–	-0.00%	-64.9%	-71.9%	-83.5%	-93.3%	-94.2%

TABLE VI
COMPARISON OF RWL WITH SR, ODR, DR, MR, AND METAHEURISTICS (META)

Design	Init	SR	ODR	DR	MR	Meta
AES	31873	32517 (+2.02%)	32637 (+2.40%)	32898 (+3.22%)	32995 (+3.52%)	33065 (+3.74%)
M0	27670	28201 (+1.92%)	28271 (+2.17%)	28470 (+2.89%)	28728 (+3.82%)	28805 (+4.10%)
JPEG	101000	104562 (+3.53%)	104657 (+3.62%)	105550 (+4.50%)	107699 (+6.63%)	108173 (+7.10%)
VGA	208155	212186 (+1.94%)	212905 (+2.28%)	214169 (+2.89%)	217492 (+4.49%)	216856 (+4.18%)
MPEG	38896	39640 (+1.91%)	39799 (+2.32%)	39950 (+2.71%)	40594 (+4.37%)	40512 (+4.15%)
Avg.	+0.00%	+2.26%	+2.56%	+3.24%	+4.57%	+4.66%

TABLE VII
COMPARISON OF RUNTIME (S) WITH SR, ODR, DR, MR, AND METAHEURISTICS (META)

Design	SR	ODR	DR	MR	Meta
AES	32	8	59	162	348
M0	22	8	51	174	214
JPEG	325	50	344	776	2153
VGA	493	51	386	713	1658
MPEG	30	11	86	137	234

size. A larger design of millions of instances may see more benefits.

For *Vt* and *drive* distribution, the results show similar *step* reduction percentage, demonstrating the robustness of our optimization. Fig. 16 shows the layouts of placements before and after MR optimization.

We also investigate the improvement achieved by our MR optimization over SR, DR optimization, and previous works. We compare MR optimization to: 1) SR optimization (also to match [5], [16]); 2) ordered DR (ODR) optimization (to match [15]); and 2) DR optimization. For 1), we use the proposed methodology in Section III and fix the locations of all multi-height cells. We note that our SR implementation is equivalent to [5] and [16], supporting neighboring cell swapping and cell flipping with the adaptation of NDE. In SR, we use the same displacement range and reordering range as in DR, while using the default HPWL weighting factor $\gamma = 0.0001$ (HPWL weighting factor is not considered in the work of [7]). For 2), we simply run our DR optimization with zero reordering range to achieve an ODR equivalent to [15]. For 3), we use the proposed methodology in Section IV. The comparisons of #*steps*, RWL, and runtime are shown in Tables V–VII, respectively. For design blocks with fewer double-height cells, SR performance is competitive with that of ODR. However, for design blocks with more double-height cells, ODR is significantly better (up to 21% more *step* reduction) than SR due to movable double-height cells. The results show that DR effectively reduces the diffusion *steps* by around half compared to SR, and by around 40% compared to ODR. On average, DR has 11.6% more *step* reduction than ODR, and 17.7% more than SR, with respect to the initial number of diffusion *steps*. This suggests the importance of supporting movable and reorderable double-height cells, as there will be substantial benefits.

D. Metaheuristics

We have also explored several metaheuristics to assess: 1) the *step* reduction achievable by invoking multiple optimization iterations, as well as 2) potential improved tradeoffs between *step* reduction and degradation from initial placement (in terms of HPWL). First, we investigate the maximum *step* reduction versus the number of iterations. To explore the maximum benefits of *step* reduction, we invoke the MR optimization several times. Since the MR optimization is for every two rows, e.g., row 1 and 2 in a window, row 3 and 4 in the next window, we can shift the window by one row and run again if we can further improve the solution quality. In our experiments, we alternatively align/unalign the optimization window with double-height cells, with aligned window in the first iteration to encourage the movement of double-height cells. We show the normalized number of *diffusion steps* and HPWL versus the number of optimization iterations (up to 8) in Fig. 17. Compared to one iteration, the second iteration removes 45 out of 152 remaining steps after the first iteration, while the remaining six iterations only reduce 13 more *steps*, at the cost of increased HPWL.

Given the above observation, we seek to obtain a better tradeoff between *step* reduction and HPWL. Since our MR optimization is not HPWL-aware, we propose to invoke both SR and MR optimization with a total "budget" of four iterations, to find the best four-iteration sequence. We explore all possible optimization sequences composed of the following three configurations: 1) SR HPWL-aware; 2) MR aligned with double-height cells; and 3) MR unaligned with double-height cells. We report the optimized number of *steps*, along

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

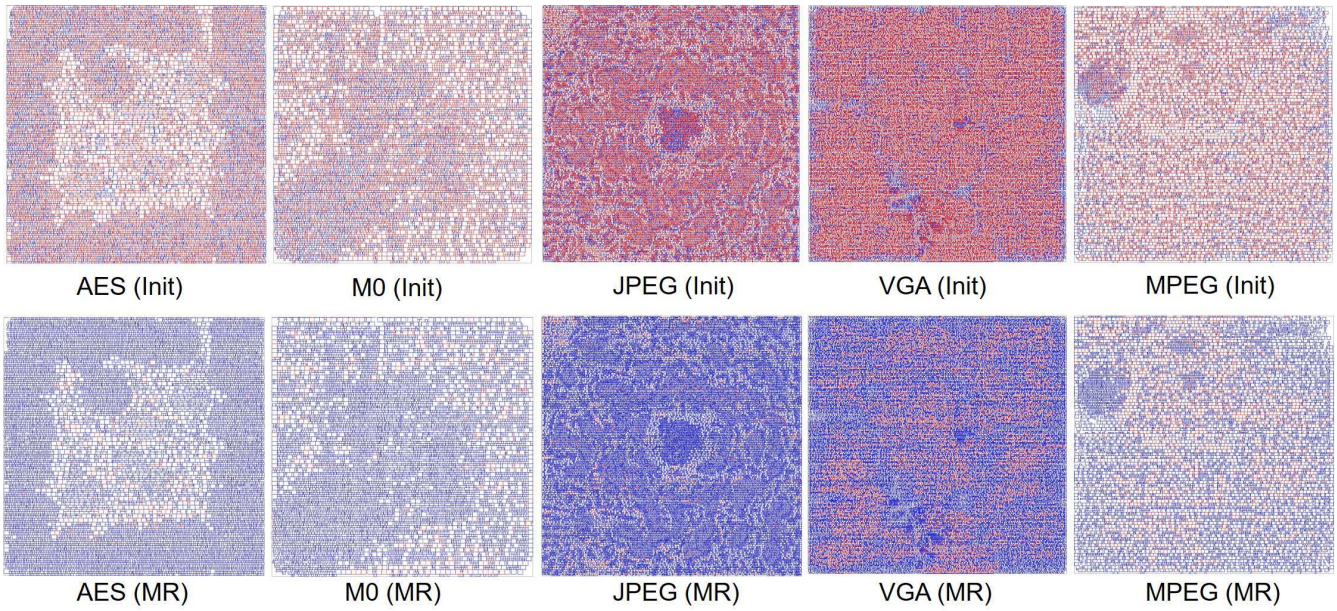


Fig. 16. Layouts of placements before (Init) and after (MR) our MR optimization. Red color indicates cell instances with diffusion *steps* and blue color indicates cell instances without diffusion *steps*.

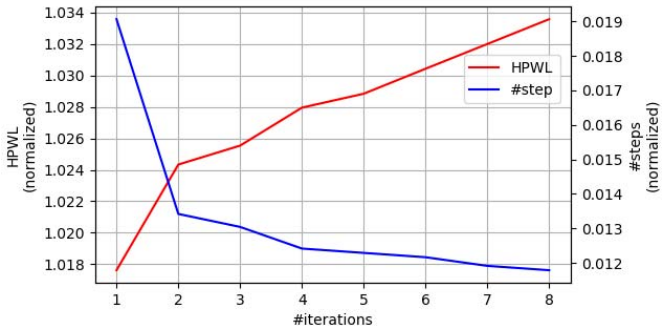


Fig. 17. #*Steps* (normalized) and HPWL (normalized) versus #iterations in metaheuristic optimization.

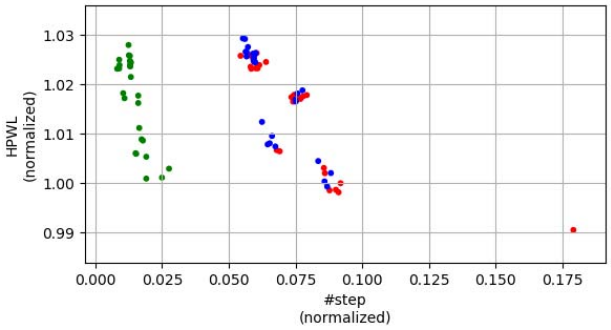


Fig. 18. #Steps versus HPWL in metaheuristic optimization. Red (resp. green and blue) dots represent metaheuristic iterations that start with configuration 1 (resp. configuration 2 and configuration 3).

with HPWL, in Fig. 18. We can see that the configuration for the first iteration largely determines the optimized number of *steps*. The first iteration should be 2) to obtain better *step* reduction. Also, the optimization should finish with 1) for better HPWL. We report the metaheuristic results in Tables V–VII.

E. Performance Improvement Using Intentional Steps

Similar in spirit to [12], we explore the possibility of improving design performance with *intentional steps*—i.e., using filler cells that create an *intentional step* to the neighboring timing-critical functional cell so as to improve the timing of that functional cell.[7] In the cost function, we use a third weighting factor δ to represent the benefit of an *intentional step* to a timing-critical cell. We sweep δ from 0 to -2 with a step size of -0.1. We select 5% of all cells as timing-critical cells and perform optimization using all design blocks. The results are shown in Fig. 19. We use *orig.opt* to represent the results with $\delta = 0$, and *time.opt* to represent the results with $\delta = -0.3$. Compared to $\delta = 0$, we achieve up to 5× increase in #*filler-induced steps* incident to timing-critical cells when $\delta = -0.3$, at the cost of slightly increased #*nonfiller-induced steps* to nontiming-critical cells. This translates to up to 2.13 *steps* per timing-critical cell after *time.opt*, compared to 0.42 *steps* after *orig.opt*. Overall, we can still decrease total *steps* by more than 70%, showing the effectiveness of our algorithm. We note that as we add more *intentional steps* to timing-critical cells, we leave a smaller solution space for nontiming-critical cells. Thus, *time.opt* generates more *steps* to nontiming-critical cells. We furthermore observe that as δ decreases, the #*intentional steps* that we can achieve approaches a limit, as shown in Fig. 20. This may

[7] An *intentional inter-cell step* may increase/decrease the drive strength of the function cell, e.g., a *step* adjacent to a PFET may decrease the drive strength while a *step* adjacent to an NFET may increase the drive strength. Here, instead of using a filler cell to match diffusion heights for both the NFET and the PFET of the function cell (to reduce #*steps*), we create a filler-induced *intentional step* by matching the diffusion height for only the PFET, thus increasing the drive strength for the NFET. We note that exact timing and power impacts and tradeoffs will vary with STI processes.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
DESIGN INFORMATION AND EXPERIMENT RESULTS FOR ICCAD-2017 BENCHMARK [2]. DISTRIBUTION OF SINGLE-HEIGHT, DOUBLE-HEIGHT, TRIPLE-HEIGHT, AND QUADRUPLE-HEIGHT CELLS ARE SHOWN IN COLUMNS 1×H, 2×H, 3×H, AND 4×H, RESPECTIVELY

design	#inst	cell types %				*#steps*			Runtime
		1×H	2×H	3×H	4×H	Init	Final	(Δ%)	(sec)
des_perf_b_md1	∼11K	94.80	5.20	0.00	0.00	57806	3781	(-93.46%)	361.3
des_perf_b_md2	∼11K	90.47	6.02	2.01	1.50	70733	7494	(-89.41%)	232.8
edit_dist_1_md1	∼13K	90.31	6.12	2.04	1.53	74351	6019	(-91.90%)	420.9
edit_dist_a_md2	∼13K	90.31	6.12	2.04	1.53	76657	8074	(-89.47%)	417.8
fft_2_md2	∼ 3K	89.62	6.56	2.18	1.64	22040	3789	(-82.81%)	53.2
fft_a_md2	∼ 3K	89.57	6.59	2.19	1.65	10960	606	(-94.47%)	136.4
fft_a_md3	∼ 3K	93.42	2.19	2.19	2.19	11631	372	(-96.80%)	78.1
pci_bridge32_a_md1	∼ 3K	90.39	6.07	2.02	1.52	17284	1429	(-91.73%)	83.8
des_perf_1	∼11K	100.00	0.00	0.00	0.00	73202	3516	(-95.20%)	488.7
des_perf_a_md1	∼11K	95.66	4.34	0.00	0.00	64624	3060	(-95.26%)	307.3
des_perf_a_md2	∼11K	96.99	1.00	1.00	1.00	64346	4793	(-92.55%)	315.9
edit_dist_a_md3	∼13K	93.88	2.04	2.04	2.04	78560	11100	(-85.87%)	258.9
pci_bridge32_a_md2	∼ 3K	85.51	7.08	4.05	3.37	21435	6235	(-70.91%)	71.2
pci_bridge32_b_md1	∼ 3K	90.39	6.07	2.02	1.52	14988	1070	(-92.86%)	68.1
pci_bridge32_b_md2	∼ 3K	96.97	1.01	1.01	1.01	13812	488	(-96.47%)	135.0
pci_bridge32_b_md3	∼ 3K	94.94	1.01	2.02	2.02	14929	1193	(-92.01%)	84.2

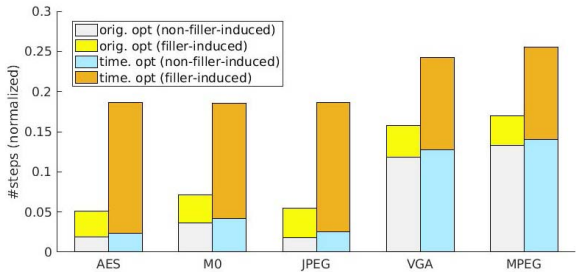


Fig. 19. Comparison of #filler-induced *steps* and total #*steps* for all design blocks before (*orig.opt*, $\delta = 0$) and after (*time.opt*, $\delta = -0.3$) using *intentional steps*.

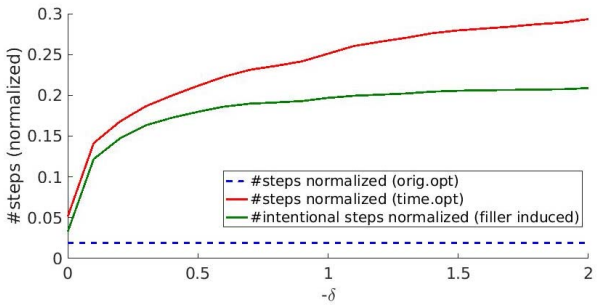


Fig. 20. Sensitivity of filler-induced *steps* to δ. Testcase: AES.

help set expectations for benefits that might be derived from a more comprehensive, timing-aware flow (which we leave for future work).

F. ICCAD-2017 Benchmark Results

We apply our MR DP-based optimization to winning solutions from the ICCAD-2017 contest [2] only considering row and site alignments, but not considering constraints, including maximum cell movement, cell edge spacing, pin access, pin shorts, and fence regions from the contest. The input legalized placements for all benchmark testcases are from the first-place team's solutions in ICCAD-2017 contest, except pci_bridge32_a_md1 and pci_bridge32_a_md2, for which we use the second-place team's solutions (because the first-place team's solutions for these two testcases have cells placed outside of the die boundary). We keep the same P/G alignment as in the input placement. We apply *rand* fin height assignment methodology with the above-mentioned 1:3:6 ratio for 2, 3 and 4 fins, respectively. The results are shown in Table VIII. For all ICCAD-2017 benchmark testcases, we achieve up to 96.8% reduction in #*steps*.

VII. CONCLUSION

In this paper, we have presented an *optimal* DP-based SR/DR/MR detailed placement methodology to minimize diffusion *steps* in sub-10 nm VLSI, for improved yield and mitigation of NDE. This paper achieves several improvements as compared to the previous works: 1) optimal DP with support of a richer set of cell movements, i.e., flipping, relocating, and enhanced reordering; 2) optimal MR DP *with support of movable and reorderable double-height cells*; and 3) a novel performance improvement technique using *intentional steps*. The proposed techniques achieve up to 98% reduction of inter-cell diffusion *steps*, with scalable runtime and high die utilization in an N7 node enablement. Open directions for future research include: 1) a more comprehensive timing-aware optimization flow that is capable of addressing the NDE while minimizing placement disturbance; 2) extension to other LDE-aware optimizations [25], [27]; 3) integration with other detailed placement objectives; and 4) speedup techniques with regard to the reordering range.

ACKNOWLEDGMENT

The authors would like to thank K. Han and H. Lee for their contribution in the work of [7].

REFERENCES

[1] D. C. Chen *et al.*, "Compact modeling solution of layout dependent effect for FinFET technology," in *Proc. ICMTS*, 2015, pp. 110–115.
[2] N. K. Darav, I. S. Bustany, A. Kennings, and R. Mamidi, "ICCAD-2017 CAD contest in multi-deck standard cell legalization and benchmarks," in *Proc. ICCAD*, Irvine, CA, USA, 2017, pp. 867–871.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.

[3] P. Debacker *et al.*, "Vertical M1 routing-aware detailed placement for congestion and wirelength reduction in sub-10nm nodes," in *Proc. DAC*, Austin, TX, USA, 2017, pp. 1–6.
[4] S. Dobre, A. B. Kahng, and J. Li, "Mixed cell-height implementation for improved design quality in advanced nodes," in *Proc. ICCAD*, Austin, TX, USA, 2015, pp. 854–860.
[5] Y. Du and M. D. F. Wong, "Optimization of standard cell based detailed placement for 16nm FinFET process," in *Proc. DATE*, 2014, pp. 1–6.
[6] J. V. Faricelli, "Layout-dependent proximity effects in deep nanoscale CMOS," in *Proc. CICC*, San Jose, CA, USA, 2010, pp. 1–8.
[7] C. Han *et al.*, "Optimal multi-row detailed placement for yield and model-hardware correlation improvements in sub-10nm VLSI," in *Proc. ICCAD*, Irvine, CA, USA, 2017, pp. 667–674.
[8] K. Han, A. B. Kahng, and H. Lee, "Scalable detailed placement legalization for complex sub-14nm constraints," in *Proc. ICCAD*, Austin, TX, USA, 2015, pp. 867–873.
[9] D. Hill, "Method and system for high speed detailed placement of cells within an integrated circuit design," U.S. Patent 6 370 673, 2002.
[10] S.-W. Hur and J. Lillis, "Mongrel: Hybrid techniques for standard cell placement," in *Proc. ICCAD*, San Jose, CA, USA, 2000, pp. 165–170.
[11] A. B. Kahng, I. L. Markov, and S. Reda, "On legalization of row-based placements," in *Proc. GLSVLSI*, 2004, pp. 214–219.
[12] A. B. Kahng, P. Sharma, and R. O. Topaloglu, "Exploiting STI stress for performance," in *Proc. ICCAD*, San Jose, CA, USA, 2007, pp. 83–90.
[13] A. B. Kahng, P. Tucker, and A. Zelikovsky, "Optimization of linear placements for wirelength minimization with free sites," in *Proc. ASP-DAC*, 1999, pp. 241–244.
[14] S. Li and C.-K. Koh, "Mixed integer programming models for detailed placement," in *Proc. ISPD*, 2012, pp. 87–94.
[15] Y. Lin *et al.*, "MrDP: Multiple-row detailed placement of heterogeneous-sized cells for advanced nodes," in *Proc. ICCAD*, Austin, TX, USA, 2016, pp. 1–8.
[16] Y. Lin, B. Yu, B. Xu, and D. Z. Pan, "Triple patterning aware detailed placement toward zero cross-row middle-of-line conflict," in *Proc. ICCAD*, Austin, TX, USA, 2015, pp. 396–403.
[17] S.-K. Oh, "Standard cell library, method of using the same, and method of designing semiconductor integrated circuit," U.S. Patent US20 160 055 283.
[18] H.-C. Ou, K.-H. Tseng, J.-Y. Liu, I.-P. Wu, and Y.-W. Chang, "Layout-dependent-effects-aware analytical analog placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 8, pp. 1243–1254, Aug. 2016.
[19] M. Pan, N. Viswanathan, and C. Chu, "An efficient and effective detailed placement algorithm," in *Proc. ICCAD*, San Jose, CA, USA, 2005, pp. 48–55.
[20] M. Tarabbia, A. Mittal, and N. Hindawy, "Forming FinFET cell with fin tip and resulting device," U.S. Patent US20 150 137 203.
[21] H. Tian, Y. Du, H. Zhang, Z. Xiao, and M. D. F. Wong, "Triple patterning aware detailed placement with constrained pattern assignment," in *Proc. ICCAD*, San Jose, CA, USA, 2014, pp. 116–123.
[22] C.-H. Wang *et al.*, "An effective legalization algorithm for mixed-cell-height standard cells," in *Proc. ASP-DAC*, 2017, pp. 450–455.
[23] G. Wu and C. Chu, "Detailed placement algorithm for VLSI design with double-row height standard cells," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 8, pp. 1569–1573, Sep. 2016.
[24] R. Xie, K.-Y. Lim, M. G. Sung, and R. R.-H. Kim, "Methods of forming single and double diffusion breaks on integrated circuit products comprised of FinFET devices and the resulting products," U.S. Patent US9 412 616, 2016.
[25] S. Yang *et al.*, "10nm high performance mobile SoC design and technology co-developed for performance, power, and area scaling," in *Proc. VLSI Technol.*, 2017, pp. T70–T71.
[26] B. Yu *et al.*, "Methodology for standard cell compliance and detailed placement for triple patterning lithography," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 5, pp. 726–739, May 2015.
[27] P. Zhao *et al.*, "Influence of stress induced CT local layout effect (LLE) on 14nm FinFET," in *Proc. VLSI Technol.*, 2017, pp. T228–T229.
[28] *Model-Hardware Correlation Team*, Samsung Electron. Co., Ltd., Seoul, South Korea, Nov. 2016.
[29] *Cadence Innovus User Guide*. Accessed: Nov. 28, 2017. [Online]. Available: http://www.cadence.com
[30] *LEF/DEF Reference 5.7*. Accessed: Nov. 28, 2017. [Online]. Available: http://www.si2.org/openeda.si2.org/projects/lefdef
[31] *Si2 OpenAccess*. Accessed: Nov. 28, 2017. [Online]. Available: http://www.si2.org/?page=69
[32] *OpenCores: Open Source IP-Cores*. Accessed: Nov. 28, 2017. [Online]. Available: http://www.opencores.org
[33] OpenMP Architecture Review Board. *OpenMP Application Program Interface, Version 4.0*. Accessed: Nov. 28, 2017. [Online]. Available: http://www.openmp.org/
[34] *Synopsys Design Compiler User Guide*. Accessed: Nov. 28, 2017. [Online]. Available: http://www.synopsys.com



Changho Han received the B.S. degree in electrical and electronic engineering from KAIST, Daejeon, South Korea, in 2001.

He is a Principal Engineer leading the Model-Hardware Correlation Team, Samsung Electronics, Hwaseong, South Korea.



Andrew B. Kahng (M'03–SM'07–F'10) received the Ph.D. degree in computer science from the University of California at San Diego, La Jolla, CA, USA.

He is a Professor with the Computer Science Engineering Department and the Electrical and Computer Engineering Department, University of California at San Diego. His current research interests include IC physical design, design-manufacturing interface, combinatorial optimization, and technology roadmapping.



Lutong Wang (S'16) received the B.S. degree in microelectronics from Tsinghua University, Beijing, China, in 2014 and the M.S. degree in electrical and computer engineering from the University of California at San Diego, La Jolla, CA, USA, in 2016, where he is currently pursuing the Ph.D. degree.

His current research interests include physical design implementation and DFM methodologies.



Bangqi Xu (S'16) received the B.S. degree in electrical engineering from the University of Michigan, Ann Arbor, MI, USA in 2015 and the M.S. degree in electrical and computer engineering from the University of California at San Diego, La Jolla, CA, USA, in 2017, where he is currently pursuing the Ph.D. degree.

His current research interests include detailed placement, PDN optimization, and machine learning.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:39:16 UTC from IEEE Xplore. Restrictions apply.